

AI ASSISTED CODING

ASSIGNMENT-11.2

NAME: Varshitha. G

HTNO: 2303A51103

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
    pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

```
1  # generate a code to demonstrate stack and its operations with docst
2  class Stack:
3      def __init__(self):
4          self.stack = []
5      def push(self, item):
6          """
7          Push an item onto the stack.
8          """
9          self.stack.append(item)
10         print(f'Pushed {item} to stack')
11     def pop(self):
12         """
13         Pop an item from the stack.
14         """
15         if not self.is_empty():
16             item = self.stack.pop()
17             print(f'Popped {item} from stack')
18             return item
19         else:
20             print('Stack is empty. Cannot pop.')
21     def peek(self):
22         """
23         Peek at the top item of the stack.
24         """
25         if not self.is_empty():
26             item = self.stack[-1]
27             print(f'Top item is {item}')
28             return item
29         else:
30             print('Stack is empty. Cannot peek.')
31     def is_empty(self):
32         """
33         Check if the stack is empty.
34         """
35         return len(self.stack) == 0
36     def size(self):
37         """
38         Get the size of the stack
```

```

"""
return len(self.stack)
self.stack.append(item)
print(f'Pushed {item} to stack')
def pop(self):
    if not self.is_empty():
        item = self.stack.pop()
        print(f'Popped {item} from stack')
        return item
    else:
        print('Stack is empty. Cannot pop.')
def peek(self):
    if not self.is_empty():
        item = self.stack[-1]
        print(f'Top item is {item}')
        return item
    else:
        print('Stack is empty. Cannot peek.')
def is_empty(self):
    return len(self.stack) == 0
def size(self):
    return len(self.stack)
# Demonstration of stack operations
if __name__ == "__main__":
    stack = Stack()
    stack.push(10)
    stack.push(20)
    stack.push(30)
    stack.peak()
    stack.pop()
    stack.peak()
    print(f'Stack size: {stack.size()}')
    stack.pop()
    stack.pop()
    stack.pop() # Attempt to pop from an empty stack
    stack.peak() # Attempt to peek from an empty stack

```

instance NoneType()

The type of the None s

```

Stack size: 2
Popped 20 from stack
Popped 10 from stack
Stack is empty. Cannot pop.
Stack is empty. Cannot peek.
Stack size: 2
Popped 20 from stack
Stack size: 2
Popped 20 from stack
Popped 10 from stack
Stack is empty. Cannot pop.
Stack is empty. Cannot peek.
PS C:\Users\varsh\OneDrive\ドキュメント\third\ATAC>

```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```

76 # generate a code to demonstrate queue and its operations
77
78 class Queue:
79     def __init__(self):
80         self.queue = []
81     def enqueue(self, item):
82         """
83         Add an item to the end of the queue.
84         """
85         self.queue.append(item)
86         print(f'Enqueued {item} to queue')
87     def dequeue(self):
88         """
89         Remove an item from the front of the queue.
90         """
91         if not self.is_empty():
92             item = self.queue.pop(0)
93             print(f'Dequeued {item} from queue')
94             return item
95         else:
96             print('Queue is empty. Cannot dequeue.')
97     def peek(self):
98         """
99         Peek at the front item of the queue.
100         """
101         if not self.is_empty():
102             item = self.queue[0]
103             print(f'Front item is {item}')
104             return item
105         else:
106             print('Queue is empty. Cannot peek.')
107     def is_empty(self):
108         """
109         Check if the queue is empty.
110         """
111         return len(self.queue) == 0
112     def size(self):
113         """
114         Get the size of the queue.
115         """
116         return len(self.queue)
117

```

```

# Demonstration of queue operations
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(10)
    queue.enqueue(20)
    queue.enqueue(30)
    queue.peek()
    queue.dequeue()
    queue.peek()
    print(f'Queue size: {queue.size()}')
    queue.dequeue()
    queue.dequeue()
    queue.dequeue() # Attempt to dequeue from an empty queue
    queue.peek() # Attempt to peek from an empty queue

```

```

Queue size: 2
Dequeued 20 from queue
Dequeued 30 from queue
Queue is empty. Cannot dequeue.
Queue is empty. Cannot peek.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation.

```

4 # generate a code to demonstrate linked list and its operations with documentation
5 class Node:
6     def __init__(self, data):
7         self.data = data
8         self.next = None
9 class LinkedList:
10    def __init__(self):
11        self.head = None
12    def append(self, data):
13        """
14        Append a new node with the given data to the end of the linked list.
15        """
16        new_node = Node(data)
17        if not self.head:
18            self.head = new_node
19            print(f'Appended {data} as head of the linked list')
20            return
21        last_node = self.head
22        while last_node.next:
23            last_node = last_node.next
24        last_node.next = new_node
25        print(f'Appended {data} to the end of the linked list')
26    def prepend(self, data):
27        """
28        Prepend a new node with the given data to the beginning of the linked list.
29        """
30        new_node = Node(data)
31        new_node.next = self.head
32        self.head = new_node
33        print(f'Prepended {data} to the beginning of the linked list')
34    def delete_with_value(self, data):
35        """
36        Delete the first node with the specified data from the linked list.
37        """
38        if not self.head:
39            print('Linked list is empty. Cannot delete.')
40            return
41        if self.head.data == data:
42            self.head = self.head.next
43            print(f'Deleted node with value {data} from linked list')
44            return

```

```

2         print(f'Node with value {data} not found in linked list')
3     def display(self):
4         """
5         Display all nodes in the linked list.
6         """
7         nodes = []
8         current_node = self.head
9         while current_node:
10            nodes.append(current_node.data)
11            current_node = current_node.next
12        print('Linked List:', ' -> '.join(map(str, nodes)))
13    # Demonstration of linked list operations
14    if __name__ == "__main__":
15        linked_list = LinkedList()
16        linked_list.append(10)
17        linked_list.append(20)
18        linked_list.append(30)
19        linked_list.display()
20        linked_list.prepend(5)
21        linked_list.display()
22        linked_list.delete_with_value(20)
23        linked_list.display()
24        linked_list.delete_with_value(40) # Attempt to delete a non-existent value
25        linked_list.display()
26

```

```

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData
ers/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment_14.py
Appended 10 as head of the linked list
Appended 20 to the end of the linked list
Appended 30 to the end of the linked list
Linked List: 10 -> 20 -> 30
Prepended 5 to the beginning of the linked list
Linked List: 5 -> 10 -> 20 -> 30
Deleted node with value 20 from linked list
Linked List: 5 -> 10 -> 30
Node with value 40 not found in linked list
Linked List: 5 -> 10 -> 30
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

- BST implementation with recursive insert and traversal methods.

```

class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None
    def insert(self, data):
        """Insert a new node with the given data into the binary tree."""
        if not self.root:
            self.root = TreeNode(data)
            print(f'Inserted {data} as root of the binary tree')
        else:
            self._insert_recursive(self.root, data)
    def _insert_recursive(self, node, data):
        if data < node.data:
            if node.left is None:
                node.left = TreeNode(data)
                print(f'Inserted {data} to the left of {node.data}')
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = TreeNode(data)
                print(f'Inserted {data} to the right of {node.data}')
            else:
                self._insert_recursive(node.right, data)
    def inorder_traversal(self):
        """Perform an inorder traversal of the binary tree and return a list of values."""
        return self._inorder_recursive(self.root)
    def _inorder_recursive(self, node):
        if node is None:
            return []
        return self._inorder_recursive(node.left) + [node.data] + self._inorder_recursive(node.right)

if __name__ == "__main__":
    binary_tree = BinaryTree()
    binary_tree.insert(10)
    binary_tree.insert(5)
    binary_tree.insert(15)
    binary_tree.insert(3)

```

```

C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>python 100_assignment1.py
● Inserted 10 as root of the binary tree
  Inserted 5 to the left of 10
  Inserted 15 to the right of 10
  Inserted 3 to the left of 5
  Inserted 7 to the right of 5
  Inserted 12 to the left of 15
  Inserted 18 to the right of 15
  Inorder Traversal: [3, 5, 7, 10, 12, 15, 18]
○ PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

```
pass
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

```

57 class HashTable:
58     def __init__(self, size=10):
59         """Initialize the hash table with a specified size."""
60         self.size = size
61         self.table = [[] for _ in range(size)] # Create a list of empty lists for chaining
62     def _hash(self, key):
63         """Generate a hash for the given key."""
64         return hash(key) % self.size
65     def insert(self, key, value):
66         """Insert a key-value pair into the hash table."""
67         index = self._hash(key)
68         # Check if the key already exists and update it
69         for i, (k, v) in enumerate(self.table[index]):
70             if k == key:
71                 self.table[index][i] = (key, value) # Update existing key
72                 print(f'Updated key {key} with value {value}')
73                 return
74         # If the key does not exist, add a new key-value pair
75         self.table[index].append((key, value))
76         print(f'Inserted key {key} with value {value}')
77     def search(self, key):
78         """Search for a value by its key in the hash table."""
79         index = self._hash(key)
80         for k, v in self.table[index]:
81             if k == key:
82                 print(f'Found key {key} with value {v}')
83                 return v # Return the value associated with the key
84         print(f'Key {key} not found')
85         return None # Key not found
86     def delete(self, key):
87         """Delete a key-value pair from the hash table."""
88         index = self._hash(key)
89         for i, (k, v) in enumerate(self.table[index]):
90             if k == key:
91                 del self.table[index][i] # Remove the key-value pair
92                 print(f'Deleted key {key}')
93                 return
94         print(f'Key {key} not found. Cannot delete.')
95 # Demonstration of hash table operations
96 if __name__ == "__main__":
97     hash_table = HashTable()
98     hash_table.insert('name', 'Alice')
99     hash_table.insert('age', 30)
100    hash_table.insert('city', 'New York')
101    hash_table.search('name')

```

```

Inserted key name with value Alice
Inserted key age with value 30
Inserted key city with value New York
Found key name with value Alice
Found key age with value 30
Key country not found
Deleted key age
Key age not found
Key country not found. Cannot delete.

```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

```
310 class Graph:
311     def __init__(self):
312         """Initialize the graph with an empty adjacency list."""
313         self.graph = {}
314     def add_vertex(self, vertex):
315         """Add a vertex to the graph."""
316         if vertex not in self.graph:
317             self.graph[vertex] = []
318             print(f'Added vertex {vertex}')
319         else:
320             print(f'Vertex {vertex} already exists')
321     def add_edge(self, vertex1, vertex2):
322         """Add an edge between two vertices in the graph."""
323         if vertex1 in self.graph and vertex2 in self.graph:
324             self.graph[vertex1].append(vertex2)
325             self.graph[vertex2].append(vertex1) # For undirected graph
326             print(f'Added edge between {vertex1} and {vertex2}')
327         else:
328             print('One or both vertices not found. Cannot add edge.')
329     def display(self):
330         """Display the graph as an adjacency list."""
331         for vertex, edges in self.graph.items():
332             print(f'{vertex}: {edges}')
333 # Demonstration of graph operations
334 if __name__ == "__main__":
335     graph = Graph()
336     graph.add_vertex('A')
337     graph.add_vertex('B')
338     graph.add_vertex('C')
339     graph.add_edge('A', 'B')
340     graph.add_edge('A', 'C')
341     graph.add_edge('B', 'C')
342     graph.display()
```

PROBLEMS	OUTPUT	TERMINAL	PORTS
> TERMINAL <pre>PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe C:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment_14.py Added vertex A Added vertex B Added vertex C Added edge between A and B Added edge between A and C Added edge between B and C A: ['B', 'C'] B: ['A', 'C']</pre>			

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

class PriorityQueue:

pass

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```

346 import heapq
347 class PriorityQueue:
348     def __init__(self):
349         """Initialize the priority queue with an empty list."""
350         self.elements = []
351     def enqueue(self, item, priority):
352         """Add an item to the priority queue with a given priority."""
353         heapq.heappush(self.elements, (priority, item))
354         print(f'Enqueued item {item} with priority {priority}')
355     def dequeue(self):
356         """Remove and return the item with the highest priority (lowest value)."""
357         if not self.is_empty():
358             priority, item = heapq.heappop(self.elements)
359             print(f'Dequeued item {item} with priority {priority}')
360             return item
361         else:
362             print('Priority queue is empty. Cannot dequeue.')
363             return None
364     def is_empty(self):
365         """Check if the priority queue is empty."""
366         return len(self.elements) == 0
367     def display(self):
368         """Display the items in the priority queue."""
369         print('Priority Queue:', [(priority, item) for priority, item in self.elements])
370 # Demonstration of priority queue operations
371 if __name__ == "__main__":
372     pq = PriorityQueue()
373     pq.enqueue('task1', 3)
374     pq.enqueue('task2', 1)
375     pq.enqueue('task3', 2)
376     pq.display()

```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ TERMINAL

```

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python310/python.exe C:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment_14.py
Dequeued item task2 with priority 1
Priority Queue: [(2, 'task3'), (3, 'task1')]
Dequeued item task3 with priority 2
Dequeued item task1 with priority 3
Priority queue is empty. Cannot dequeue.
Priority Queue: []
Priority queue is empty. Cannot dequeue.
Priority Queue: []
Priority Queue: []

```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`.

Sample Input Code:

```
class DequeDS:
```

```
pass
```

Expected Output:

- Insert and remove from both ends with docstrings.

```

386 from collections import deque
387 class Deque:
388     def __init__(self):
389         """Initialize the deque with an empty deque object."""
390         self.deque = deque()
391     def insert_front(self, item):
392         """Insert an item at the front of the deque."""
393         self.deque.appendleft(item)
394         print(f'Inserted {item} at the front of the deque')
395     def insert_rear(self, item):
396         """Insert an item at the rear of the deque."""
397         self.deque.append(item)
398         print(f'Inserted {item} at the rear of the deque')
399     def remove_front(self):
400         """Remove and return an item from the front of the deque."""
401         if not self.is_empty():
402             item = self.deque.popleft()
403             print(f'Removed {item} from the front of the deque')
404             return item
405         else:
406             print('Deque is empty. Cannot remove from front.')
407             return None
408     def remove_rear(self):
409         """Remove and return an item from the rear of the deque."""
410         if not self.is_empty():
411             item = self.deque.pop()
412             print(f'Removed {item} from the rear of the deque')
413             return item
414         else:
415             print('Deque is empty. Cannot remove from rear.')
416             return None
417     def is_empty(self):
418         """Check if the deque is empty."""
419         return len(self.deque) == 0
420     def display(self):
421         """Display the items in the deque."""
422         print('Deque:', list(self.deque))
423 # Demonstration of deque operations
424 if __name__ == "__main__":
425     my_deque = Deque()
426     my_deque.insert_rear(10)
427     my_deque.insert_rear(20)
428     my_deque.insert_front(5)
429     my_deque.display()
430     my_deque.remove_front()

```

```

Inserted 10 at the rear of the deque
Inserted 20 at the rear of the deque
Inserted 5 at the front of the deque
Deque: [5, 10, 20]
Removed 5 from the front of the deque
Deque: [10, 20]
Removed 20 from the rear of the deque
Deque: [10]
Removed 10 from the front of the deque
Deque is empty. Cannot remove from rear.
Deque: []

```

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> █

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

439 # generate a code for Cafeteria Order Queue - Serve students in the order they arrive using a queue data
440 class CafeteriaOrderQueue:
441     def __init__(self):
442         """Initialize the cafeteria order queue with an empty list."""
443         self.orders = []
444     def enqueue(self, student_name):
445         """Add a student's order to the end of the queue."""
446         self.orders.append(student_name)
447         print(f'Enqueued order for {student_name}')
448     def dequeue(self):
449         """Serve the next student in the order they arrived (remove from the front of the queue)."""
450         if not self.is_empty():
451             student_name = self.orders.pop(0)
452             print(f'Served {student_name}')
453             return student_name
454         else:
455             print('No orders to serve. The queue is empty.')
456             return None
457     def is_empty(self):
458         """Check if there are no orders in the queue."""
459         return len(self.orders) == 0
460     def display(self):
461         """Display the current orders in the queue."""
462         print('Current Orders in Queue:', self.orders)
463 # Demonstration of cafeteria order queue operations
464 if __name__ == "__main__":
465     order_queue = CafeteriaOrderQueue()
466     order_queue.enqueue('Alice')
467     order_queue.enqueue('Bob')
468     order_queue.enqueue('Charlie')
469     order_queue.display()
470     order_queue.dequeue()
471     order_queue.display()
472     order_queue.dequeue()
473     order_queue.dequeue()
474     order_queue.dequeue() # Attempt to serve from an empty queue
475     order_queue.display() # Display the empty queue
476

```

```

ive/ドキュメント/third/AIAC/lab_assignment_14.py
Enqueued order for Alice
Enqueued order for Bob
Enqueued order for Charlie
Current Orders in Queue: ['Alice', 'Bob', 'Charlie']
Served Alice
Current Orders in Queue: ['Bob', 'Charlie']
Served Bob
Served Charlie
No orders to serve. The queue is empty.
Current Orders in Queue: []
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Feature	Chosen Data Structure	Justification
Student Attendance Tracking	Hash Table	A hash table allows fast insertion and lookup of student entries/exits by ID. It ensures efficient daily logging and quick verification of whether a student is currently on campus.

Feature	Chosen Data Structure	Justification
Event Registration System	Binary Search Tree (BST)	A BST supports ordered storage of participants, enabling quick search, insertion, and removal. It is ideal for managing event participants where alphabetical or ID-based ordering is useful.
Library Book Borrowing	Priority Queue	A priority queue can track books by due dates, ensuring that the most urgent returns (earliest due dates) are prioritized. This helps librarians manage overdue books efficiently.
Bus Scheduling System	Graph	Bus routes and stop connections naturally form a graph, where stops are nodes and routes are edges. Graph traversal algorithms can be used to find shortest paths or optimize scheduling.
Cafeteria Order Queue	Queue	A queue ensures students are served in the order they arrive (FIFO principle). This matches the real-world cafeteria process where fairness and order are maintained.

A Queue is the best fit because it follows the First-In-First-Out (FIFO) principle. This means students are served in the exact order they arrive, ensuring fairness and efficiency. Just like in a real cafeteria line, the first student to place an order will be the first to get served.

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

6 class HospitalManagementSystem:
7     def __init__(self):
8         """Initialize the hospital management system with an empty patient queue."""
9         self.patient_queue = []
10    def check_in(self, patient_name):
11        """Register a patient and add them to the end of the queue."""
12        self.patient_queue.append(patient_name)
13        print(f'Checked in patient: {patient_name}')
14    def treat_patient(self):
15        """Treat the next patient in the order they arrived (remove from the front of the queue)."""
16        if not self.is_empty():
17            patient_name = self.patient_queue.pop(0)
18            print(f'Treated patient: {patient_name}')
19            return patient_name
20        else:
21            print('No patients to treat. The queue is empty.')
22            return None
23    def is_empty(self):
24        """Check if there are no patients in the queue."""
25        return len(self.patient_queue) == 0
26    def display_queue(self):
27        """Display the current patients in the queue."""
28        print('Current Patients in Queue:', self.patient_queue)
29    # Demonstration of hospital management system operations
30    if __name__ == "__main__":
31        hospital_system = HospitalManagementSystem()
32        hospital_system.check_in('John Doe')
33        hospital_system.check_in('Jane Smith')
34        hospital_system.check_in('Emily Davis')
35        hospital_system.display_queue()
36        hospital_system.treat_patient()
37        hospital_system.display_queue()
38        hospital_system.treat_patient()
39        hospital_system.treat_patient()
40        hospital_system.treat_patient() # Attempt to treat from an empty queue
41        hospital_system.display_queue() # Display the empty queue

```

```

Checked in patient: John Doe
Checked in patient: Jane Smith
Checked in patient: Emily Davis
Current Patients in Queue: ['John Doe', 'Jane Smith', 'Emily Davis']
Treated patient: John Doe
Current Patients in Queue: ['Jane Smith', 'Emily Davis']
Treated patient: Jane Smith
Treated patient: Emily Davis
No patients to treat. The queue is empty.
Current Patients in Queue: []
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Feature	Data Structure	Justification
Patient Check-In System	Queue	Patients are treated in the order they arrive, which follows the FIFO (First-In-First-Out) principle. A queue ensures fair and sequential handling.
Emergency Case Handling	Priority Queue	Critical patients must be treated first. A priority queue allows assigning urgency levels so higher-priority patients are handled before others.
Medical Records Storage	Hash Table	Patient details can be quickly retrieved using their unique ID. Hash tables provide $O(1)$ average-time lookup.
Doctor Appointment Scheduling	Linked List	Appointments can be stored in sorted order by time. A linked list allows easy insertion and traversal in chronological order.
Hospital Room Navigation	Graph	Wards and rooms are interconnected. A graph represents these connections, enabling efficient navigation and shortest path finding.

Patients must be treated in the exact order they arrive. A queue ensures fairness by following the FIFO principle, so no one skips ahead unless marked as an emergency case (which would use a priority queue instead).

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- ☐ Stack
- ☐ Queue
- ☐ Priority Queue
- ☐ Linked List
- ☐ Binary Search Tree (BST)
- ☐ Graph
- ☐ Hash Table

o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

524 class TrafficSignalQueue:
525     def __init__(self):
526         """Initialize the traffic signal queue with an empty list."""
527         self.vehicle_queue = []
528     def arrive(self, vehicle_number):
529         """Add a vehicle to the end of the queue when it arrives at the traffic signal."""
530         self.vehicle_queue.append(vehicle_number)
531         print(f'Vehicle {vehicle_number} arrived at the traffic signal')
532     def pass_signal(self):
533         """Allow the next vehicle in the order they arrived to pass through the traffic signal (remove from the front of the queue)."""
534         if not self.is_empty():
535             vehicle_number = self.vehicle_queue.pop(0)
536             print(f'Vehicle {vehicle_number} passed through the traffic signal')
537             return vehicle_number
538         else:
539             print('No vehicles to pass. The queue is empty.')
540             return None
541     def is_empty(self):
542         """Check if there are no vehicles in the queue."""
543         return len(self.vehicle_queue) == 0
544     def display_queue(self):
545         """Display the current vehicles waiting at the traffic signal."""
546         print('Current Vehicles in Queue:', self.vehicle_queue)
547 # Demonstration of traffic signal queue operations
548 if __name__ == "__main__":
549     traffic_queue = TrafficSignalQueue()
550     traffic_queue.arrive('ABC123')
551     traffic_queue.arrive('XYZ789')
552     traffic_queue.arrive('LMN456')
553     traffic_queue.display_queue()
554     traffic_queue.pass_signal()
555     traffic_queue.display_queue()
556     traffic_queue.pass_signal()

```

PROBLEMS OUTPUT TERMINAL PORTS

2d

```

> ✓ TERMINAL
Vehicle ABC123 arrived at the traffic signal
Vehicle XYZ789 arrived at the traffic signal
Vehicle LMN456 arrived at the traffic signal
Current Vehicles in Queue: ['ABC123', 'XYZ789', 'LMN456']
Vehicle ABC123 passed through the traffic signal
Current Vehicles in Queue: ['XYZ789', 'LMN456']
Vehicle XYZ789 passed through the traffic signal
Vehicle LMN456 passed through the traffic signal
No vehicles to pass. The queue is empty.
Current Vehicles in Queue: []

```

Feature	Data Structure	Justification
Traffic Signal Queue	Queue	Vehicles wait at signals in the order they arrive. A queue follows the FIFO principle, ensuring fair passage.
Emergency Vehicle Priority Handling	Priority Queue	Ambulances and fire trucks must be prioritized. A priority queue ensures vehicles with higher urgency are processed first.
Vehicle Registration Lookup	Hash Table	Vehicle details can be quickly retrieved using registration numbers. Hash tables provide O(1) average lookup time.
Road Network Mapping	Graph	Roads and intersections form a network. A graph represents connections, enabling shortest path and route planning.

Feature	Data Structure	Justification
Parking Slot Availability	Binary Search Tree (BST)	Parking slots can be tracked as available/occupied. A BST allows efficient searching, insertion, and deletion of slots by number.

Vehicles at a traffic signal must move in the order they arrive. A queue ensures fairness and follows the FIFO principle, so no vehicle skips ahead unless marked as an emergency (which would use a priority queue).

Task Description #12: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- ☐ Stack
- ☐ Queue
- ☐ Priority Queue
- ☐ Linked List
- ☐ Binary Search Tree (BST)
- ☐ Graph
- ☐ Hash Table
- ☐ Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

562 class ShoppingCart:
563     def __init__(self):
564         """Initialize the shopping cart with an empty list of products."""
565         self.cart = []
566     def add_product(self, product_name):
567         """Add a product to the shopping cart."""
568         self.cart.append(product_name)
569         print(f'Added {product_name} to the shopping cart')
570     def remove_product(self, product_name):
571         """Remove a product from the shopping cart if it exists."""
572         if product_name in self.cart:
573             self.cart.remove(product_name)
574             print(f'Removed {product_name} from the shopping cart')
575         else:
576             print(f'Product {product_name} not found in the shopping cart. Cannot remove.')
577     def display_cart(self):
578         """Display the current products in the shopping cart."""
579         print('Current Shopping Cart:', self.cart)
580 # Demonstration of shopping cart management operations
581 if __name__ == "__main__":
582     shopping_cart = ShoppingCart()
583     shopping_cart.add_product('Laptop')
584     shopping_cart.add_product('Smartphone')
585     shopping_cart.add_product('Headphones')
586     shopping_cart.display_cart()
587     shopping_cart.remove_product('Smartphone')
588     shopping_cart.display_cart()
589     shopping_cart.remove_product('Tablet') # Attempt to remove a non-existent product
590     shopping_cart.display_cart() # Display the current state of the cart
591

```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ TERMINAL

```

ive/ドキュメント/third/ATAC/lab_assignment_14.py
Added Laptop to the shopping cart
Added Smartphone to the shopping cart
Added Headphones to the shopping cart
Current Shopping Cart: ['Laptop', 'Smartphone', 'Headphones']
Removed Smartphone from the shopping cart
Current Shopping Cart: ['Laptop', 'Headphones']
Product Tablet not found in the shopping cart. Cannot remove.
Current Shopping Cart: ['Laptop', 'Headphones']

```

Feature	Data Structure	Justification
Shopping Cart Management	Linked List	Products can be added or removed dynamically. A linked list supports efficient insertion and deletion without shifting elements.
Order Processing System	Queue	Orders must be processed in the order they are placed. A queue ensures FIFO (First-In-First-Out) handling.
Top-Selling Products Tracker	Priority Queue	Products can be ranked by sales count. A priority queue ensures the highest-selling products are always accessible first.
Product Search Engine	Hash Table	Products can be quickly retrieved using product IDs. Hash tables provide O(1) average lookup time.

Feature	Data Structure	Justification
Delivery Route Planning	Graph	Warehouses and delivery locations form a network. A graph represents connections and supports shortest path algorithms.

A shopping cart needs dynamic addition and removal of products. A linked list is ideal because it allows efficient insertion and deletion without shifting elements, unlike arrays.

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
594 class ShipmentProcessing:
595     def __init__(self):
596         """Initialize the shipment processing system with an empty list of shipments."""
597         self.shipments = []
598     def add_shipment(self, shipment_id):
599         """Add a shipment to the processing queue."""
600         self.shipments.append(shipment_id)
601         print(f'Added shipment {shipment_id} to the processing queue')
602     def process_shipment(self):
603         """Process the next shipment in the order they were added (remove from the front of the queue)."""
604         if not self.is_empty():
605             shipment_id = self.shipments.pop(0)
606             print(f'Processed shipment {shipment_id}')
607             return shipment_id
608         else:
609             print('No shipments to process. The queue is empty.')
610             return None
611     def is_empty(self):
612         """Check if there are no shipments in the processing queue."""
613         return len(self.shipments) == 0
614     def display_queue(self):
615         """Display the current shipments in the processing queue."""
616         print('Current Shipments in Queue:', self.shipments)
617 # Demonstration of shipment processing operations
618 if __name__ == "__main__":
619     shipment_processing = ShipmentProcessing()
620     shipment_processing.add_shipment('SHIP001')
621     shipment_processing.add_shipment('SHIP002')
622     shipment_processing.add_shipment('SHIP003')
623     shipment_processing.display_queue()
624     shipment_processing.process_shipment()
625     shipment_processing.display_queue()
626     shipment_processing.process_shipment()
```

PROBLEMS OUTPUT **TERMINAL** PORTS

> **TERMINAL**

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/py
ive/ドキュメント/third/AIAC/lab_assignment_14.py
○ Current Shipments in Queue: ['SHIP002', 'SHIP003']
Processed shipment SHIP002
Processed shipment SHIP003
No shipments to process. The queue is empty.
No shipments to process. The queue is empty.
Current Shipments in Queue: []
```

Feature	Data Structure	Justification
Shipment Processing	Queue	Shipments are processed in the order they arrive. A queue ensures FIFO (First-In-First-Out) handling.
Urgent Shipment Priority	Priority Queue	Urgent shipments must be handled first. A priority queue allows assigning urgency levels so critical shipments are processed before normal ones.
Product Lookup by Code	Hash Table	Products can be quickly retrieved using their unique code. Hash tables provide O(1) average lookup time.
Warehouse Network Connectivity	Graph	Warehouses and routes form a connected network. A graph represents these connections and supports shortest path algorithms for efficient delivery.

Feature	Data Structure	Justification
Inventory Add/Remove Operations	Linked List	Inventory needs dynamic addition and removal of products. A linked list supports efficient insertion and deletion without shifting elements.

Shipments must be processed in the order they arrive. A queue ensures fairness and follows the FIFO principle, so no shipment is skipped unless marked urgent (which would be handled by a priority queue).

Task Description #14: Smart Banking Management System – Data

Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.
3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

2 class CustomerServiceTokenSystem:
5     self.token_queue = []
6     def take_token(self, customer_name):
7         """Assign a token to a customer and add them to the end of the queue."""
8         self.token_queue.append(customer_name)
9         print(f'{customer_name} took a token and is added to the queue')
10    def serve_customer(self):
11        """Serve the next customer in the order they arrived (remove from the front of the queue)."""
12        if not self.is_empty():
13            customer_name = self.token_queue.pop(0)
14            print(f'Served customer: {customer_name}')
15            return customer_name
16        else:
17            print('No customers to serve. The queue is empty.')
18            return None
19    def is_empty(self):
20        """Check if there are no customers in the queue."""
21        return len(self.token_queue) == 0
22    def display_queue(self):
23        """Display the current customers waiting in the queue."""
24        print('Current Customers in Queue:', self.token_queue)
25
26 # Demonstration of customer service token system operations
27 if __name__ == "__main__":
28     token_system = CustomerServiceTokenSystem()
29     token_system.take_token('Alice')
30     token_system.take_token('Bob')
31     token_system.take_token('Charlie')
32     token_system.display_queue()
33     token_system.serve_customer()
34     token_system.display_queue()
35     token_system.serve_customer()
36     token_system.serve_customer()

```

TERMINAL
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python...
ive/ドキュメント/third/AIAC/lab_assignment_14.py
Bob took a token and is added to the queue
Charlie took a token and is added to the queue
Current Customers in Queue: ['Alice', 'Bob', 'Charlie']
Served customer: Alice
Current Customers in Queue: ['Bob', 'Charlie']
Served customer: Bob
Served customer: Charlie
No customers to serve. The queue is empty.
Current Customers in Queue: []

Feature	Data Structure	Justification
Customer Service Token System	Queue	Customers are served in the order they arrive. A queue ensures FIFO (First-In-First-Out) handling, maintaining fairness.
Loan Approval Priority Handling	Priority Queue	High-value or urgent loans must be processed first. A priority queue allows assigning priority levels so urgent loans are handled before regular ones.
Account Information Lookup	Hash Table	Customer details can be quickly retrieved using account numbers. Hash tables provide O(1) average lookup time.

Feature	Data Structure	Justification
Transaction History Management	Linked List	Transactions are stored chronologically. A linked list allows efficient insertion at the end and traversal in order.
ATM Network Connectivity	Graph	ATMs across cities form a connected network. A graph represents these connections and supports shortest path algorithms for efficient routing.

Customers must be served in the order they arrive. A queue ensures fairness by following the FIFO principle, so no one skips ahead unless marked as urgent (which would use a priority queue).

Task Description #15: Online Learning Management System – Data Structure Selection

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.
5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

- Select the most appropriate data structure for each feature from the given list.
- Justify your selection in 2–3 sentences per feature.
- Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

- Feature → Data Structure → Justification table.
- Functional Python program with proper documentation.

```
669 class LearningManagementSystem:
670     def __init__(self):
671         """Initialize the learning management system with an empty dictionary of users."""
672         self.users = {}
673     def register_user(self, username, password):
674         """Register a new user with a username and password."""
675         if username in self.users:
676             print(f'Username {username} already exists. Please choose a different username.')
677         else:
678             self.users[username] = password
679             print(f'User {username} registered successfully.')
680     def authenticate_user(self, username, password):
681         """Authenticate a user by validating their credentials."""
682         if username in self.users and self.users[username] == password:
683             print(f'User {username} authenticated successfully.')
684             return True
685         else:
686             print(f'Authentication failed for user {username}. Invalid credentials.')
687             return False
688 if __name__ == "__main__":
689     lms = LearningManagementSystem()
690     lms.register_user("alice", "password123")
691     lms.register_user("bob", "password456")
692     lms.authenticate_user("alice", "password123")
693     lms.authenticate_user("charlie", "password789")
```

PROBLEMS OUTPUT **TERMINAL** PORTS

> **TERMINAL**

```
User alice registered successfully.
User bob registered successfully.
User alice authenticated successfully.
Authentication failed for user charlie. Invalid credentials.
```

Feature	Data Structure	Justification
Student Login Authentication	Hash Table	User credentials (username, password) need quick validation. A hash table allows $O(1)$ average-time lookup for authentication.
Assignment Submission Queue	Queue	Submissions must be processed in the order they are uploaded. A queue ensures FIFO (First-In-First-Out) handling.
Top Performer Ranking	Priority Queue	Students should be ranked by scores. A priority queue ensures the highest-scoring students are always accessible first.
Course Prerequisite Structure	Graph	Courses often depend on other courses. A graph represents subject dependencies and supports traversal for prerequisite checks.
Undo Feature in Online Editor	Stack	Undo operations require reversing recent actions. A stack follows LIFO (Last-In-First-Out), making it ideal for undo functionality.

Undo requires reversing the most recent action. A stack is perfect because it follows the LIFO principle, ensuring the last action performed is the first one undone.

Task Description #16: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

- Mapping table.
- Working Python implementation.

```
694 # generate a code to demonstrate an airport authority plans to implement: Passenger Check-in Queue – Process
695 class PassengerCheckInQueue:
696     def __init__(self):
697         """Initialize the passenger check-in queue with an empty list."""
698         self.passenger_queue = []
699     def check_in(self, passenger_name):
700         """Add a passenger to the end of the check-in queue."""
701         self.passenger_queue.append(passenger_name)
702         print(f'Checked in passenger: {passenger_name}')
703     def process_passenger(self):
704         """Process the next passenger in the order they arrived (remove from the front of the queue)."""
705         if not self.is_empty():
706             passenger_name = self.passenger_queue.pop(0)
707             print(f'Processed passenger: {passenger_name}')
708             return passenger_name
709         else:
710             print('No passengers to process. The queue is empty.')
711             return None
712     def is_empty(self):
713         """Check if there are no passengers in the queue."""
714         return len(self.passenger_queue) == 0
715     def display_queue(self):
716         """Display the current passengers in the check-in queue."""
717         print('Current Passengers in Queue:', self.passenger_queue)
718 # Demonstration of passenger check-in queue operations
719 if __name__ == "__main__":
720     check_in_queue = PassengerCheckInQueue()
721     check_in_queue.check_in('John Doe')
722     check_in_queue.check_in('Jane Smith')
723     check_in_queue.check_in('Emily Davis')
724     check_in_queue.display_queue()
725     check_in_queue.process_passenger()
```

PROBLEMS OUTPUT TERMINAL PORTS

> ▼ TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/py
Checked in passenger: Emily Davis
Current Passengers in Queue: ['John Doe', 'Jane Smith', 'Emily Davis']
Processed passenger: John Doe
Current Passengers in Queue: ['Jane Smith', 'Emily Davis']
Processed passenger: Jane Smith
Processed passenger: Emily Davis
No passengers to process. The queue is empty.
○ Current Passengers in Queue: []
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Feature	Data Structure	Justification
Passenger Check-In Queue	Queue	Passengers must be processed in the order they arrive. A queue ensures FIFO (First-In-First-Out) handling, maintaining fairness.
Emergency Landing Priority	Priority Queue	Aircraft in emergency situations must be prioritized. A priority queue ensures urgent landings are handled before regular ones.
Flight Information Lookup	Hash Table	Flight details can be quickly retrieved using flight IDs. Hash tables provide O(1) average lookup time.
Air Route Connectivity	Graph	Airports and routes form a connected network. A graph represents these connections and supports shortest path algorithms for efficient navigation.
Boarding Process (Cabin Storage)	Stack	Overhead luggage follows LIFO (Last-In-First-Out) since the last bag placed is the first one removed. A stack is ideal for simulating this process.

Overhead luggage storage works in a LIFO manner—the last bag placed is the first one removed. A stack perfectly models this behavior, making it the most suitable data structure.

Task Description #17: Smart Social Media Platform – Data Structure Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

- Select appropriate data structures from the list.
- Justify your choices clearly.
- Implement one selected feature as a Python program using AI-assisted code generation.

Expected Output:

- Table mapping features to data structures with justification.
- Working Python code with docstrings and comments.

```

733 class NewsFeed:
734     def __init__(self):
735         """Initialize the news feed with an empty list of posts."""
736         self.posts = []
737     def add_post(self, post_content):
738         """Add a new post to the news feed."""
739         self.posts.append(post_content)
740         print(f'Added post: {post_content}')
741     def display_feed(self):
742         """Display the news feed with posts in chronological order."""
743         print('News Feed:')
744         for post in self.posts:
745             print(post)
746 # Demonstration of news feed generation operations
747 if __name__ == "__main__":
748     news_feed = NewsFeed()
749     news_feed.add_post('Post 1: Hello World!')
750     news_feed.add_post('Post 2: Welcome to the social media platform.')
751     news_feed.add_post('Post 3: Enjoy your stay!')
752     news_feed.display_feed()
753

```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ **TERMINAL**

```

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local
ive/ドキュメント/third/AIAC/lab_assignment_14.py
Added post: Post 1: Hello World!
Added post: Post 2: Welcome to the social media platform.
Added post: Post 3: Enjoy your stay!
News Feed:
Post 1: Hello World!
Post 2: Welcome to the social media platform.
Post 3: Enjoy your stay!

```

Feature	Data Structure	Justification
News Feed Generation	Linked List	Posts are displayed chronologically. A linked list allows efficient insertion at the end and traversal in order.
Trending Hashtags Tracker	Priority Queue	Hashtags must be ranked by usage frequency. A priority queue ensures the most frequently used hashtags are always accessible first.
User Profile Search	Hash Table	User profiles can be quickly retrieved using usernames. Hash tables provide O(1) average lookup time.
Friend Network Representation	Graph	Friend connections form a network. A graph represents relationships and supports traversal for mutual connections or recommendations.

Feature	Data Structure	Justification
Room Allotment	Binary Search Tree (BST)	Rooms can be allotted based on room numbers. A BST allows efficient searching, insertion, and deletion of rooms in sorted order.
Medical Emergency Handling	Priority Queue	Emergency cases must be handled first. A priority queue ensures urgent medical needs are prioritized over normal requests.
Student Record Retrieval	Hash Table	Student details can be quickly retrieved using unique IDs. Hash tables provide $O(1)$ average lookup time.
Hostel Block Connectivity	Graph	Hostel blocks and rooms form a connected network. A graph represents these connections and supports navigation or shortest path finding.
Visitor Entry Log	Queue	Visitors are logged in the order they arrive. A queue ensures FIFO (First-In-First-Out) handling of entries.

Visitors must be logged and processed in the order they arrive. A queue ensures fairness by following the FIFO principle, so no visitor is skipped ahead.

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
 2. Fault Priority Handling
 3. Consumer Record Lookup
 4. Power Grid Connectivity
 5. Meter Reading History Tracking
- Justify your choice in 2–3 sentences per feature.

- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

786 class ComplaintHandlingSystem:
787     def __init__(self):
788         """Initialize the complaint handling system with an empty list of complaints."""
789         self.complaints = []
790     def file_complaint(self, customer_name, complaint_details):
791         """File a new complaint and add it to the list of complaints."""
792         complaint = {'customer_name': customer_name, 'complaint_details': complaint_details}
793         self.complaints.append(complaint)
794         print(f'Filed complaint from {customer_name}: {complaint_details}')
795     def resolve_complaint(self):
796         """Resolve the next complaint in the order they were filed (remove from the front of the list)."""
797         if not self.is_empty():
798             complaint = self.complaints.pop(0)
799             print(f'Resolved complaint from {complaint["customer_name"]}: {complaint["complaint_details"]}')
800             return complaint
801         else:
802             print('No complaints to resolve. The list is empty.')
803             return None
804     def is_empty(self):
805         """Check if there are no complaints in the system."""
806         return len(self.complaints) == 0
807     def display_complaints(self):
808         """Display the current complaints in the system."""
809         print('Current Complaints:')
810         for complaint in self.complaints:
811             print(f'{complaint["customer_name"]}: {complaint["complaint_details"]}')
812 if __name__ == "__main__":
813     complaint_system = ComplaintHandlingSystem()
814     complaint_system.file_complaint('Alice', 'Power outage in my area')
815     complaint_system.file_complaint('Bob', 'Billing issue with my last bill')
816     complaint_system.file_complaint('Charlie', 'Faulty meter reading')
817     complaint_system.display_complaints()
818     complaint_system.resolve_complaint()
819     complaint_system.display_complaints()
820     complaint_system.resolve_complaint()
821     complaint_system.resolve_complaint()
822     complaint_system.resolve_complaint() # Attempt to resolve from an empty list
823     complaint_system.display_complaints() # Display the empty list of complaints

```

```

Filed complaint from Alice: Power outage in my area
Filed complaint from Bob: Billing issue with my last bill
Filed complaint from Charlie: Faulty meter reading
Current Complaints:
Alice: Power outage in my area
Bob: Billing issue with my last bill
Charlie: Faulty meter reading
Resolved complaint from Alice: Power outage in my area
Current Complaints:
Bob: Billing issue with my last bill
Charlie: Faulty meter reading
Resolved complaint from Bob: Billing issue with my last bill
Resolved complaint from Charlie: Faulty meter reading
No complaints to resolve. The list is empty.
Current Complaints:

```

Feature	Data Structure	Justification
Complaint Handling	Queue	Complaints must be addressed in the order they are received. A queue ensures FIFO (First-In-First-Out) processing, maintaining fairness.
Fault Priority Handling	Priority Queue	Critical faults must be resolved first. A priority queue allows assigning urgency levels so severe faults are handled before minor ones.

Feature	Data Structure	Justification
Consumer Record Lookup	Hash Table	Consumer details can be quickly retrieved using unique IDs. Hash tables provide $O(1)$ average lookup time.
Power Grid Connectivity	Graph	Power grids and substations form a connected network. A graph represents these connections and supports shortest path or flow analysis.
Meter Reading History Tracking	Linked List	Meter readings are stored chronologically. A linked list allows efficient insertion at the end and traversal in order.

Complaints must be processed in the order they are received. A queue ensures fairness by following the FIFO principle, so no complaint is skipped ahead unless marked urgent (which would be handled by a priority queue).